

SPRINT 3 PROYECTO CRIPTOMONEDAS

Isabel Valentina Victoria

Jonathan Moya

Sebastián Acosta

Introducción:

Durante el desarrollo del Sprint 3 se implementó un sistema de recomendación basado en técnicas de *Machine Learning*, con el fin de integrar un modelo de **clustering** previamente entrenado dentro de una **API REST** construida con el framework **FastAPI**.

El objetivo fue permitir que usuarios externos pudieran interactuar con el modelo mediante solicitudes HTTP, obteniendo respuestas estructuradas en formato **JSON**. Esta implementación convierte el modelo entrenado en un servicio accesible, escalable y reutilizable, facilitando su integración con aplicaciones web, móviles o de análisis de datos.

Descripción del Desarrollo:

El trabajo se realizó utilizando el entorno de desarrollo **Visual Studio Code**, donde se configuró un entorno virtual con las dependencias necesarias para FastAPI, Pandas, scikit-learn y joblib.

Posteriormente, se creó el archivo principal de la API (app.py), encargado de inicializar el servicio, definir los endpoints y establecer la conexión con el modelo entrenado.

El modelo de *clustering* fue generado en el sprint anterior mediante el algoritmo **K-Means**, utilizando datos históricos de criptomonedas obtenidos de la API de **CryptoCompare**.

Estos datos incluían variables como precios de apertura, cierre, máximos, mínimos y volumen de transacciones.

La API se construyó con el objetivo de recibir datos relacionados con una criptomoneda, analizarlos mediante el modelo y retornar el grupo o clúster al que pertenece según su comportamiento histórico.

Endpoints implementados:

Durante el desarrollo del Sprint se diseñaron y probaron los siguientes endpoints:

- **GET /health** → Permite verificar el estado del servicio.
 - Devuelve un mensaje confirmando que la API se encuentra activa.
- **POST /predict** → Recibe los datos de una criptomoneda (precios, volumen, etc.) y utiliza el modelo de clustering para determinar a qué grupo pertenece.
 - Retorna un JSON con el número del clúster y un mensaje descriptivo.
- **GET /recommendations** → Proporciona una lista de criptomonedas recomendadas o agrupadas según los resultados del modelo.
 - Permite observar tendencias o asociaciones entre diferentes activos digitales.
- **GET /history** (*opcional*) → Muestra un registro de las predicciones realizadas durante la sesión.
 - Sirve para llevar un seguimiento del uso del sistema

Pruebas y validación:

Para la validación del servicio se realizaron pruebas a través de **Swagger UI**, una herramienta que FastAPI genera automáticamente y que permite interactuar con la API de manera visual.

API de Recomendación de Criptomonedas ^{1.0} OAS 3.1

/openapi.json

API que ofrece recomendaciones basadas en clustering de criptomonedas

default

GET	/health	Health Check	▼
POST	/predict	Predict	▼
GET	/recommendations	Get Recommendations	▼
GET	/history	Get History	▼

Schemas

HTTPValidationError	>	Expand all	object
InvestmentProfile	>	Expand all	object
ValidationError	>	Expand all	object

default

GET

/health

Health Check

^

Parameters

Cancel

No parameters

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/health' \
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/health
```

Server response

Code	Details
200	Response body <pre>{ "status": " API activa y funcionando correctamente" }</pre> Download

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/predict' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "open": 45200.5,
    "high": 46000.3,
    "low": 44900.1,
    "close": 45800.8,
    "volumeto": 1234567
  }'
```

Request URL

```
http://127.0.0.1:8000/predict
```

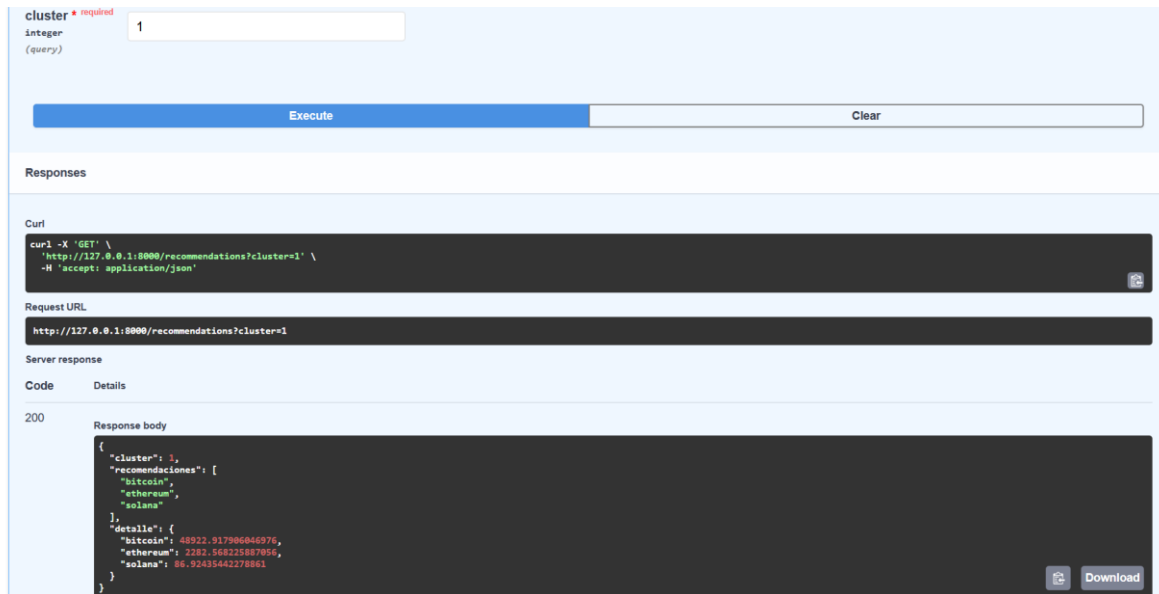
Server response

Code	Details
200	Response body <pre>{ "cluster": 1, "mensaje": "El perfil pertenece al clúster 1" }</pre> Download

Response headers

```
content-length: 59
content-type: application/json
date: Wed, 12 Nov 2025 21:11:33 GMT
server: uvicorn
```

Responses



Resultados obtenidos:

- Se logró integrar de forma completa el modelo de clustering en una API funcional.
- La API fue documentada y probada exitosamente mediante Swagger UI.
- Se estableció una estructura de rutas y respuestas estandarizada en formato JSON.
- Se garantizó la modularidad y escalabilidad del sistema, permitiendo su futura conexión con otros servicios o interfaces.

Link del github:

<https://github.com/TheSilver118/InteligenciaNegocios.git>